Heap

- It is a complete binary tree.
- each node value is greater or smaller than the node value of its children.

Shape of a Complete Binary Tree

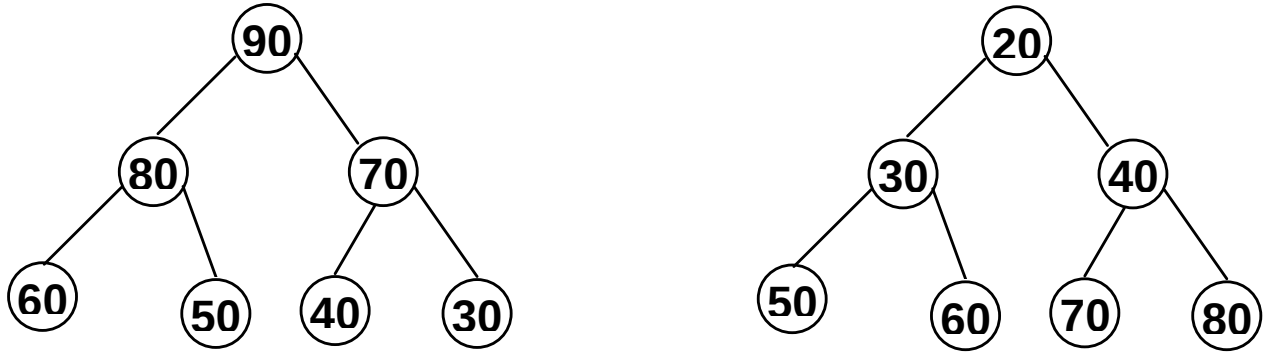


Types of Heap

- Max-heap and Min-heap
- If the node value is greater than the node value of its children, then the heap is called max heap.
- Otherwise, the heap is called min heap.

- Array implementation of a heap is efficient.

Heap [contd.]



(a): A max heap

(b): A min heap

Figure 7.12: Graphical representation of Heap

Heap Creation Process

1. Add an element as root of the heap.
2. Select second element as left child and place it in proper position by comparing it with its parent.
3. Select next element as right child and place it in proper position by comparing it with its parent.
4. Select next element as left or right child and place it in proper position by comparing it with is parent and grant parents (if any).

Heap Creation [contd.]

- We have to compare parent and child and check whether parent is smaller if smaller, swap the elements.

- By repeating the process we can have a max-heap.

Heap Creation [contd.]

- Given List: 45 28 52 25 60 70
- Create heap using the above data.

Heap Creation [contd.]

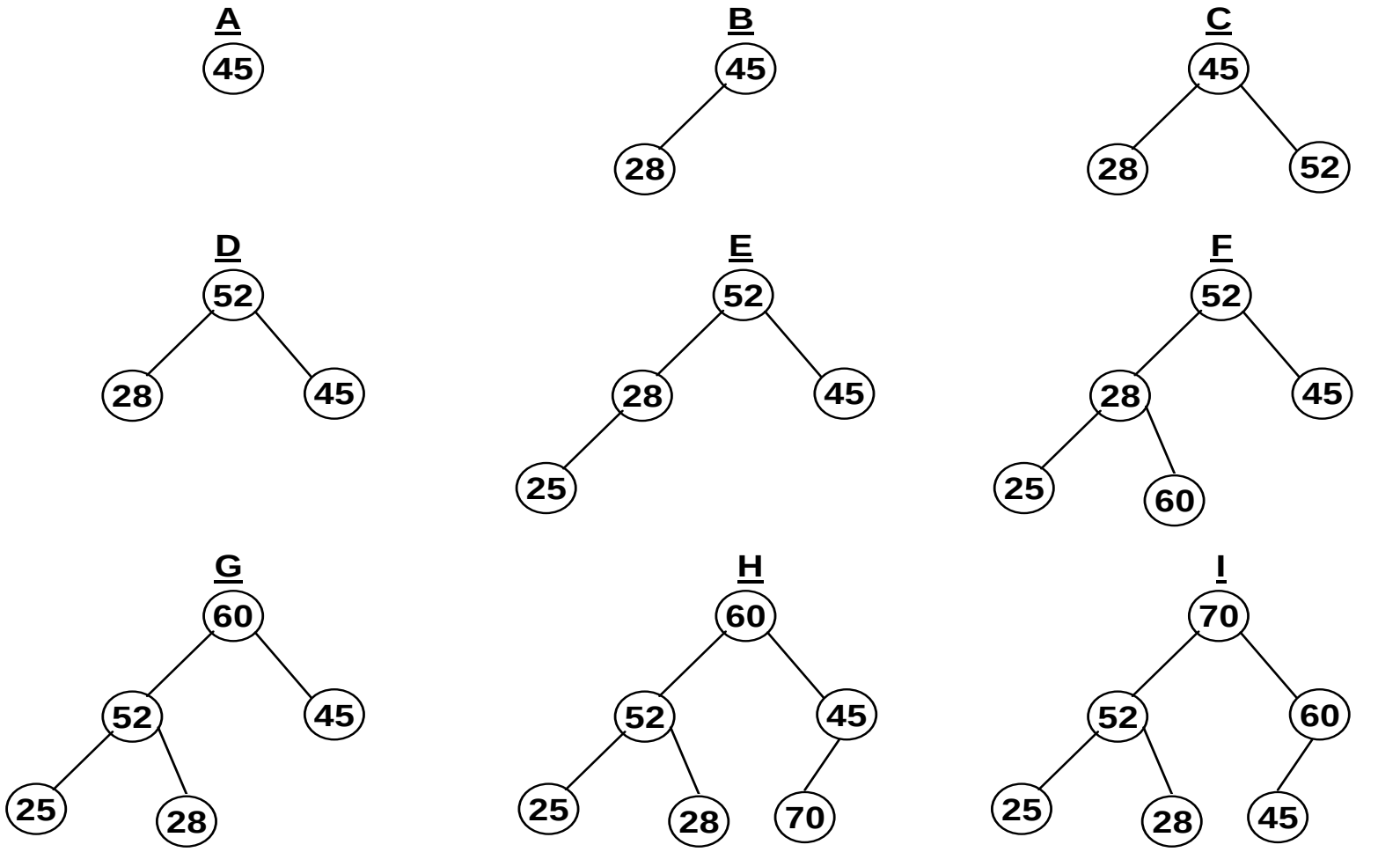


Figure 7.13: Heap creation process (pictorial view)

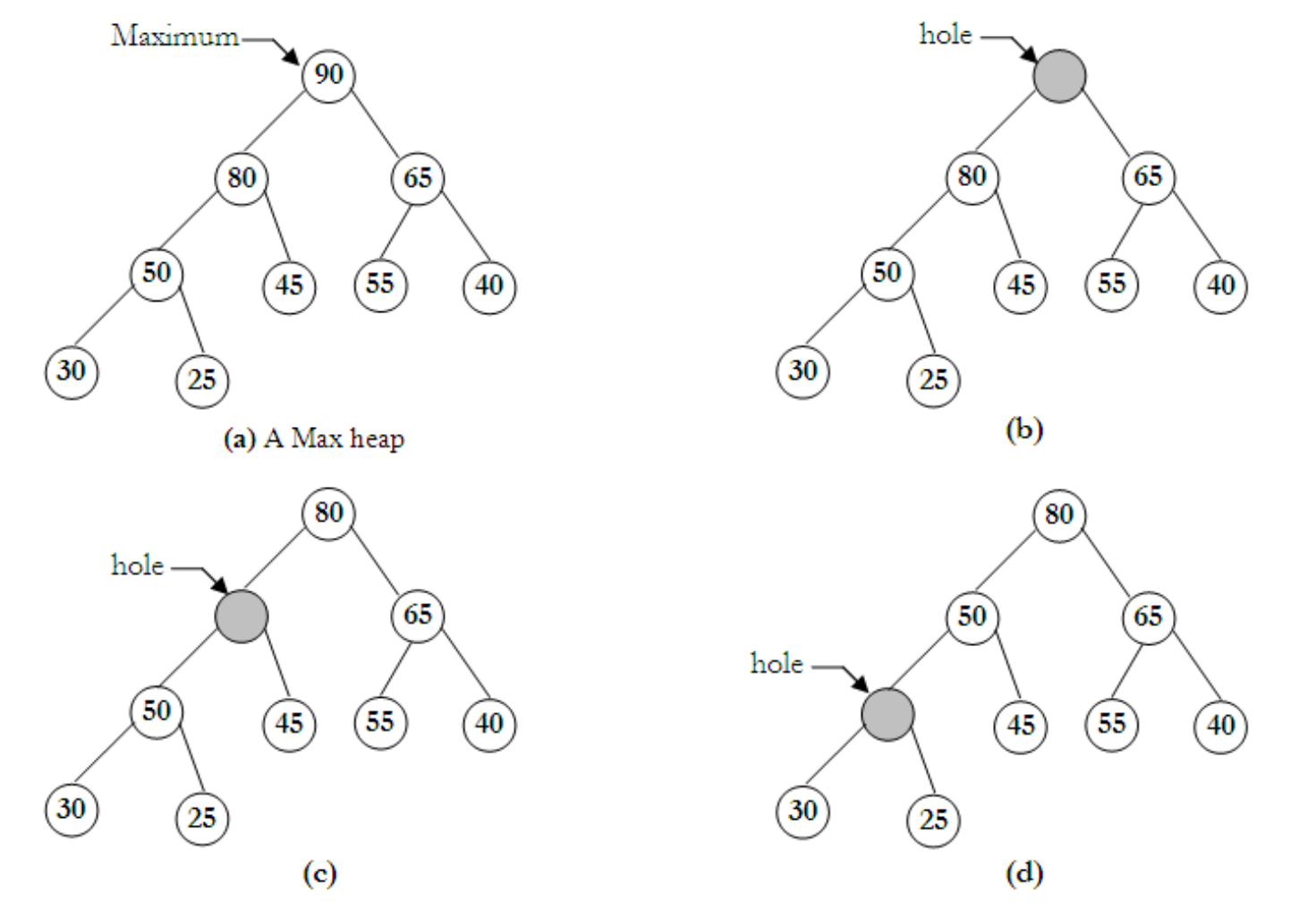
Algorithm to create a heap (pseudocode)

1. Take an array, *A*[1:*n*] and a variable, *temp*
   [we shall add data using temp one by one]
   2. for (i = 2; i<=n; ++i)
   {
   temp=a[i];
   k = *i*;
   while (*k* > 0 and *A*[*k*/2] < *temp*)
   {
   A[*k*] = *A*[*k*/2];
   k = *k*/2;
   }
   A[*k*] = *temp*;
   } // end of for
3. Output: a heap

Delete maximum from a max-heap

1. Take last data in temporary variable (temp = a[n])
2. Assume there is a hole at the root node.
3. Compare greater child of the hole with data in temp and place the greater one in the hole (root).
4. Repeat step 3 until there is no hole or the hole becomes a leaf node.
5. Place the value of temp in the hole at the last level.

Delete maximum from a max-heap [contd.]



Delete maximum from a max-heap [contd.]

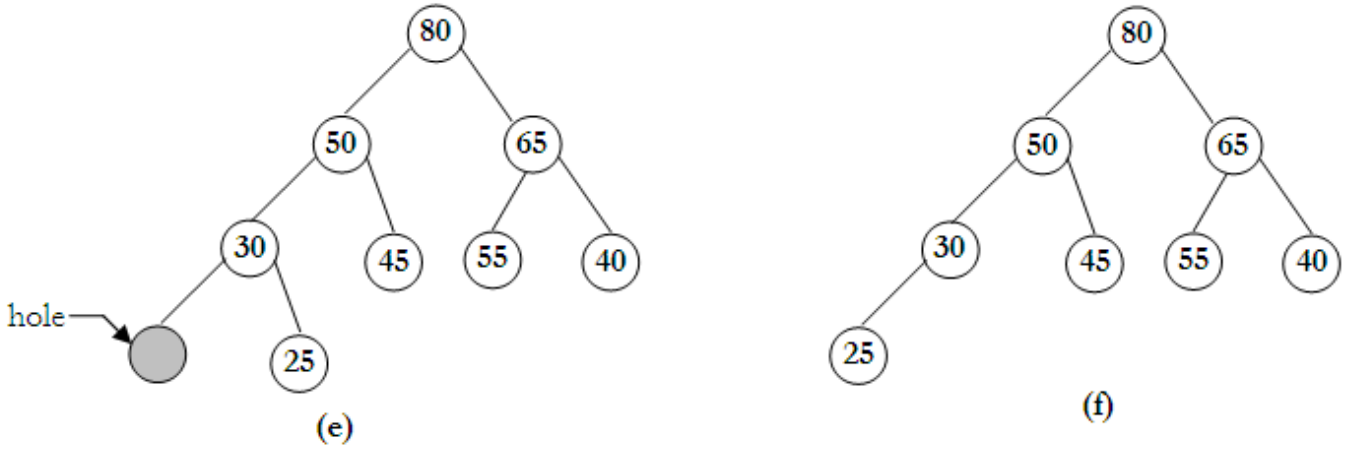


Figure 7.14: Delete the Maximum from a Max heap (pictorial view)

Algorithm to delete the Maximum
(pseudocode)

1. Input a heap, *A*[1:n]. // The heap as an array
2. *i* = 2;
3. temp= a[n];
4. while (i<=n)

```
   {
 if ((i<n) and (a[i]<a[i+1]))
 i= i+1;
If (temp>=a[i] break;
A[i/2]=a[i]; i=2*I
}
A[i/2]=temp;
```

5. Output updated heap with (n – 1) elements.

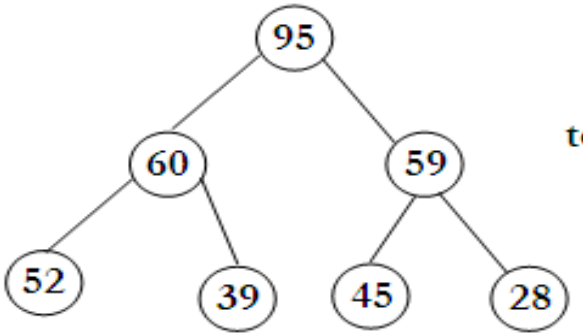
Heapsort

1. Create heap
2. Place the last node value in a temporary place
3. Place the root node value in the last place
4. Rearrange the tree except last node
5. Repeat step 2 to 4.

Heap sort in detail

1. Create a heap with *n* numbers. We can create heap using the heap creation algorithm.
2. Place the value of nth (last) node in a temporary variable.
3. Place the value of the first node (root at present) in the nth position.
4. Treat there is a hole at the root.
5. Compare the children of the hole and place the greater child in the hole.
6. Repeat step-5 until the hole is a leaf node.
7. Place the value in the temporary place in the (last) hole.
8. Consider the rest of the data (which are in a heap) except last one.
9. Repeat the step-2 to step-7 until there is only one element in the heap.

Heap sort [contd.]

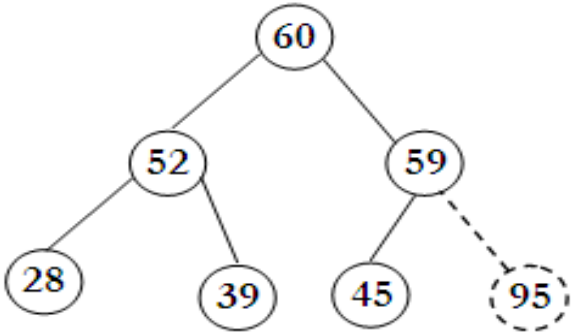


temp = 28

A Max heap with n elements

Array:	95	60	59	52	39	45	28

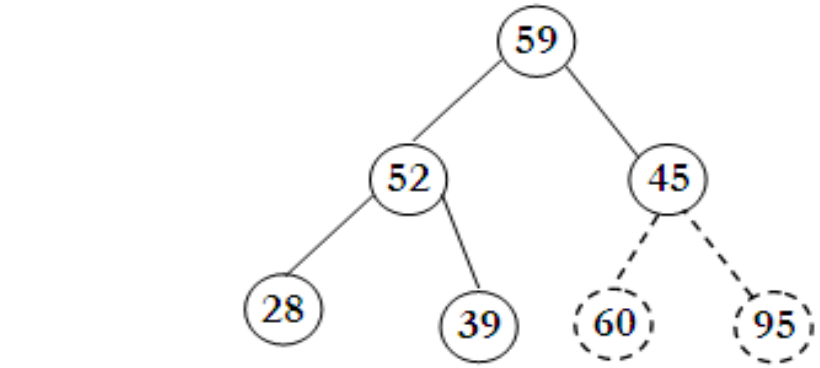
a) A heap and its corresponding array



Array:	60	52	59	28	39	45	95

b) First step of sorting process using heap

Heap sort [contd.]



Temp =45

Array:	59	52	45	28	39	60	95

c) Second step of sorting process using heap

	---	---	---	---	---	---
	---	---	---	---	---	---
	---	---	---	---	---	---

Array:	28	39	45	52	59	60	95

d) Sorted data

Figure 7.15: Pictorial view of heap sorting process

Heap Sort

1. Input an array, a[1:n] with random data
2. heapify (a, n);
3. for (k = n; k>1; --k)
   rearange (a, k);
4. Output a sorted list in a [1:n];

Here heapify and rearange are two functions. Then function heapify builds a heap and rearange function adjusts data for sorting.

Heap sort algorith

For pseudo-code of the algorithm see the algorithm of the book (new edition) page number 113.

This is the end of tree

THANK YOU…